IArch Tutorial - July 2024 Version 1.0

Ameline Alcouffe
Chahrazed Labba

April 8, 2025



Reminder: IArch is a tool that allows archaeologists to perform Explainable Artificial Intelligence (XAI) data analyzes without having specific programming expertise.

IArch is completely free, so please cite it in your scientific productions. Thanks!

C. Labba, A. Alcouffe, E. Crubézy and A. Boyer, IArch: An AI Tool for Digging Deeper into Archaeological Data, 2023 IEEE 35th International Conference on Tools with Artificial Intelligence (ICTAI), Atlanta, GA, USA, 2023, pp. 22-29, doi: 10.1109/ICTAI59109.2023.00012. [1]

This application was created as part of a doctoral scholarship from the Ministry of the University Paul Sabatier of Toulouse (France) and funding from the Lorrain Laboratory of Artificial Intelligence Research (LORIA, University of Lorraine, France).

Contents

1 Installation on Windows **4**
- 1.1 Installing Python 3.9 on Windows . . . 4
  - 1.1.1 Step-by-step instructions . . . 4
  - 1.1.2 Verifying the installation . . . 4
- 1.2 Installing GDAL and Fiona (via OSGeo4W) - easiest method . . . 5
- 1.3 Installing and running your application . . . 5
- 1.4 For future use . . . 6

2 Installation on MacOS **7**
- 2.1 Install Python 3.9.0 without affecting system Python . . . 7
  - 2.1.1 Step-by-step instructions . . . 7
- 2.2 Create and activate a virtual environment . . . 7
- 2.3 Install dependencies (requires GDAL and Rust on macOS) . . . 7
- 2.4 Final Step: Launch the application . . . 8
- 2.5 For future Use . . . 8

3 Introduction **9**
- 3.1 License and Warranty . . . 9
- 3.2 Technical Note . . . 9

4 About the App **10**

5 Uploading and Profiling page **11**

6 Classification Problem **14**
- 6.1 Functionalities and Services . . . 14
- 6.2 Advanced settings . . . 15
  - 6.2.1 Cross Validation Parameters . . . 15
  - 6.2.2 Proprotion of the Train Set . . . 16
  - 6.2.3 Random Forest Parameters . . . 16
  - 6.2.4 Support Vector Machine parameters . . . 17
  - 6.2.5 XGBoost parameters . . . 18
- 6.3 Example of Results . . . 19
  - 6.3.1 Data and Options . . . 19
  - 6.3.2 Cross-Validation Results . . . 19
  - 6.3.3 Train-Test(80-20) Results . . . 20
  - 6.3.4 Explainibility with SHAP . . . 21

7 Clustering **23**
- 7.1 Principle, Functionalities and Services . . . 23
- 7.2 Example of Results . . . 23
  - 7.2.1 Silhouette Score . . . 24
  - 7.2.2 Explainability with SHAP . . . 25

8 2025 UPDATE - Integration of social network analyses (SNA) **26**
- 8.1 Graph Creation . . . 26
- 8.2 Graph Visualization . . . 27
- 8.3 Graph Statistics . . . 28

9 Conclusion and Perspectives **29**

10 References **29**

1 Installation on Windows

If this is your first installation, it could take some time and storage on your device. We recommend having an internet connection and sufficient free storage.

Requirements :

- Python version should be at least python 3.9 (we recommand this version)
- The rest of the requirements are expressed within the file requirements.txt for windows and requirements_for_mac.txt

1.1 Installing Python 3.9 on Windows

1.1.1 Step-by-step instructions

1. Go to the official Python website: `https://www.python.org/downloads/release/python-390/`
2. Go to **Files** section
3. In the list of available files, download the Windows installer matching your system:
   - For 64-bit systems: `Windows installer (64-bit)`
   - For 32-bit systems: `Windows installer (32-bit)`
4. Once the file is downloaded, double-click it to launch the installer.
5. **Important:** Check the box that says `Add Python 3.9 to PATH` before clicking on `Install Now`.
6. Wait for the installation to complete, then click `Close`.

1.1.2 Verifying the installation

1. Open the Windows Command Prompt or the terminal (type `cmd` in the search bar and press Enter).
2. Type the following command and press Enter:

   ```
   python --version
   ```

3. If the installation was successful, the terminal will display something like:

   ```
   Python 3.9.x
   ```

4. To verify that `pip` (Python's package manager) is also installed, run:

   ```
   pip --version
   ```

   You should see a response similar to:

   ```
   pip 21.2.4 from ... (python 3.9)
   ```

1.2 Installing GDAL and Fiona (via OSGeo4W) - easiest method

Installing `gdal` and `fiona` directly via `pip` on Windows can be complicated due to binary dependencies and system-level libraries. Therefore, we recommend using the **OSGeo4W** installer, which provides precompiled packages and ensures compatibility.

1. Go to the OSGeo4W website: `https://trac.osgeo.org/osgeo4w/`
2. Download the installer `osgeo4w-setup-x86_64.exe` for 64-bit systems.
3. Launch the installer and select `Express install`.
4. Choose an installation directory (default is fine)
5. In the package selection screen, search for and select:
   - `gdal` (and its Python bindings)
   - Any other required packages (e.g., `ogr`, `proj`)
6. Complete the installation. You can now access the environment via the `OSGeo4W Shell`, where GDAL and Fiona are preconfigured and ready to use with Python 3.9.

1.3 Installing and running your application

Once OSGeo4W is installed and configured, you can install and run your Python application within the OSGeo4W Shell.

1. Open the **OSGeo4W Shell** (you can find it in the Start menu).
2. Navigate to the folder where your Python application is located. For example:

   ```
   cd C:\Users\YourName\Documents\my_app
   ```

3. Install the required dependencies using your `requirements.txt` file:

   ```
   pip install -r requirements.txt
   ```

4. If not already included, install `streamlit` separately:

   ```
   pip install streamlit
   ```

5. Once everything is installed, you can launch your Streamlit application:

   ```
   streamlit run app.py
   ```

Alternative: if Python 3.9 is already installed

If Python 3.9 is already installed on your system, follow these steps to check if the required components are available.

1. Open the Windows Command Prompt (`cmd`).
2. Verify that Python 3.9 is correctly installed:

   ```
   python --version
   ```

   Expected output:

   ```
   Python 3.9.x
   ```

3. Check if `pip` is installed:

   ```
   pip --version
   ```

4. Check if `gdal` and `fiona` are already installed:

   ```
   pip show gdal
   pip show fiona
   ```

   If both commands return information about the installed packages, then you're ready to proceed.

5. Install the dependencies:

   ```
   pip install -r requirements.txt
   ```

6. Install `streamlit` if not already listed in the requirements:

   ```
   pip install streamlit
   ```

7. Launch your application:

   ```
   streamlit run app.py
   ```

If GDAL or Fiona are not installed

If either `gdal` or `fiona` is not installed (or installation fails), we recommend using the **OSGeo4W Shell**, which provides precompiled versions of these packages and the proper environment configuration. Please refer to the previous section for instructions on how to install and use the OSGeo4W Shell.

1.4 For future use

Once everything has been installed and configured correctly using the OSGeo4W Shell, you do not need to reinstall anything for future sessions.

To launch the application again:

1. Open the **OSGeo4W Shell** (from the Start menu).
2. Navigate to the folder containing your Streamlit application using the `cd` command. For example:

   ```
   cd C:\Users\YourName\Documents\my_app
   ```

3. Then, simply run the following command:

```
streamlit run app.py
```

2 Installation on MacOS

2.1 Install Python 3.9.0 without affecting system Python

2.1.1 Step-by-step instructions

Copy and paste each line below into the Terminal (or command prompt).

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

You will be prompted to enter your password (the administrator password of the device).
Then, press **Enter** when asked.

Install `pyenv`

```
brew install pyenv
```

Wait for the installation to complete.

Add `pyenv` to your PATH

```
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.zshrc
echo 'export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.zshrc
echo 'eval "$(pyenv init --path)"' >> ~/.zshrc
source ~/.zshrc
```

Install and verify Python 3.9.0

```
pyenv install 3.9.0
pyenv versions
```

Set Python 3.9.0 locally in your app folder

```
cd my_project
```

If the app folder is located on an external drive (D: for example), write:

```
D:
```

Then,

```
cd /name_of_app_folder
```

2.2 Create and activate a virtual environment

```
pyenv local 3.9.0
python3 -m venv my_venv
source my_venv/bin/activate
```

2.3 Install dependencies (requires GDAL and Rust on macOS)

Install GDAL:

```
brew install gdal
```

Check that GDAL is properly configured:

```
which gdal-config
```

Add GDAL info to your environment:

```
export GDAL_CONFIG=$(which gdal-config)
export GDAL_VERSION=$(gdal-config --version)
```

Install Rust:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env
```

Install all dependencies from the requirements file:

```
pip install -r requirements_for_mac.txt
```

2.4 Final Step: Launch the application

Once all dependencies are installed:

```
streamlit run app.py
```

The application will open in your default browser. You can now use it offline.

2.5 For future Use

You don't need to reinstall everything. Just reactivate the environment and launch the app:

```
cd /path/to/IArch
source my_venv/bin/activate
streamlit run app.py
```

3 Introduction

Archaeologists and anthropologists often face a considerable amount of data, which makes a comprehensive and global analysis challenging. Even for modest-sized datasets, this task can be time-consuming. The IArch © application [1] aims to address this challenge by simplifying post-excavation data analysis. It provides an overview using machine learning algorithms [2, 3], thereby revealing underlying structures often unnoticed by traditional analysis methods. Another major obstacle is the frequent presence of missing data in archaeology. Although the application does not explicitly address this issue at present, it could predict some missing data with expert assistance, thereby improving the reliability and completeness of the analysis.

Three main features have been developed to meet the specific needs of archaeologists: (1) A detailed data description, including information such as correlations, the number of individuals and variables, and the characteristics of each variable. (2) Hypothesis testing to classify individuals according to user-defined categories and compare predicted data with actual data. (3) Hypothesis generation using the K-means algorithm, which creates clusters without initial biases, allowing for the discovery of new perspectives without prior prejudice.

IArch © promises to clarify results, highlight potential discoveries hidden within a constant flow of data, expedite certain analyses, and even integrate disciplines previously unfamiliar to archaeology. It provides a user-friendly interface that allows archaeologists to perform explainable artificial intelligence (XAI) [4] data analysis without requiring in-depth programming knowledge. The platform supports the entire AI process, from data preprocessing and feature selection to model application and prediction explanation using the SHapley Additive exPlanations (SHAP) library [5].

This innovative application was jointly developed by the Bird team at Lorraine University (LORIA), specializing in artificial intelligence, and researchers from Toulouse University (CAGT), experts in biological anthropology and archaeology. It is the result of a close collaboration aimed at bridging the gap between AI expertise and archaeologist needs. Designed within the framework of a biological anthropology thesis scheduled to be completed by the end of 2025, this application aims to make learning algorithms accessible to archaeologists who are not proficient in computer language.

It is worth noting that this version is a v1 and requires improvements, particularly in the implementation of new algorithms , especially those related to image recognition. The development team continues to strive to enhance the features of the application to meet the evolving needs of users.

This comprehensive manual accompanies users in using the application, providing detailed instructions for proceeding and interpreting the results.

3.1 License and Warranty

IArch is free software from a French Team. You can use it under the terms of the GNU General Public License published by the Free Software Foundation.

This application should not be used as the sole decision-making tool. It should complement other analyses conducted.

3.2 Technical Note

The application was developed in Visual Studio Code (1.91.0 version) [6] using Streamlit (1.31.1 version). [7]

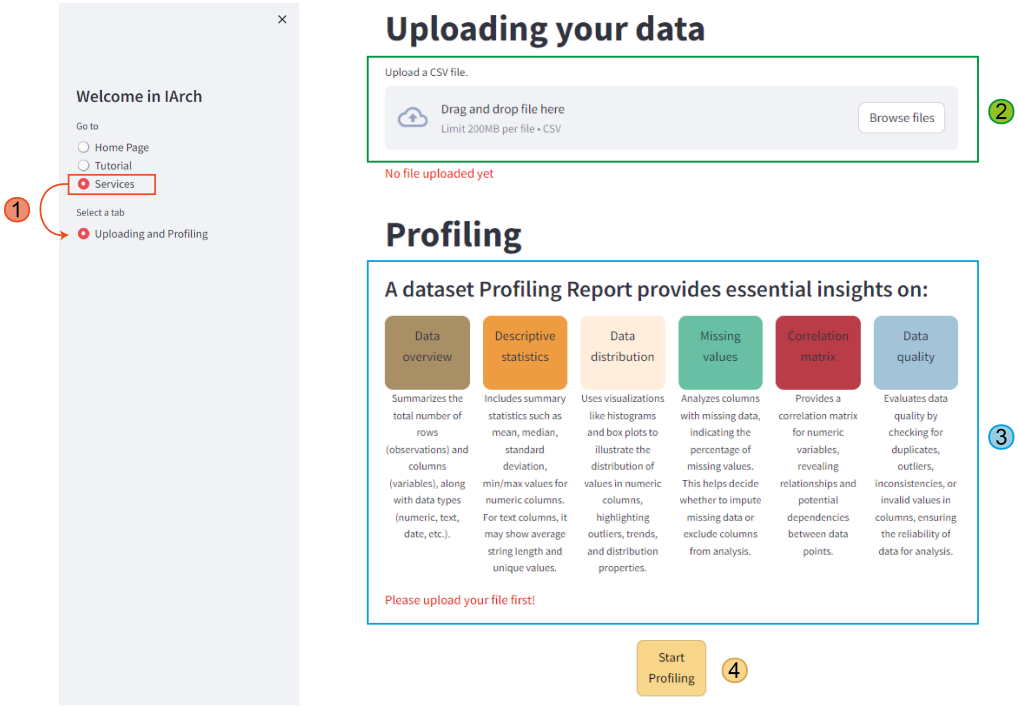
4 About the App

To load the application correctly in the window, you need to be connected to the Internet. Depending on your connection speed, the application may be more or less reactive.

This is the home page

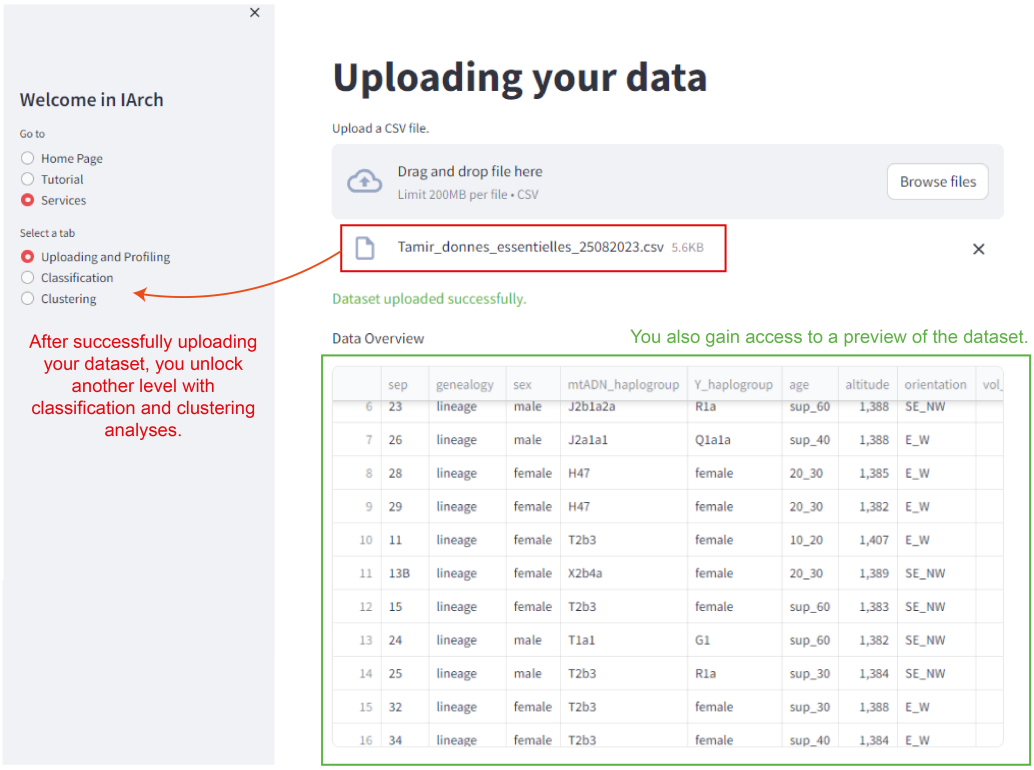


5 Uploading and Profiling page



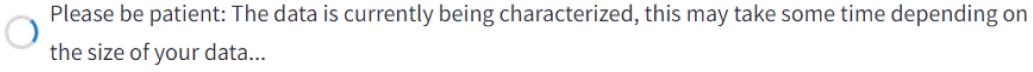
1. By clicking on the "*service*" box, you will unlock a first level that allows you to upload your data set.
2. Then, click on "***Browse files***" to upload your file. "Dataset upload successfully" indicates that your dataset has been successfully uploaded, and a preview of the uploaded dataset is provided.

Please note: your dataset must be in .csv format.

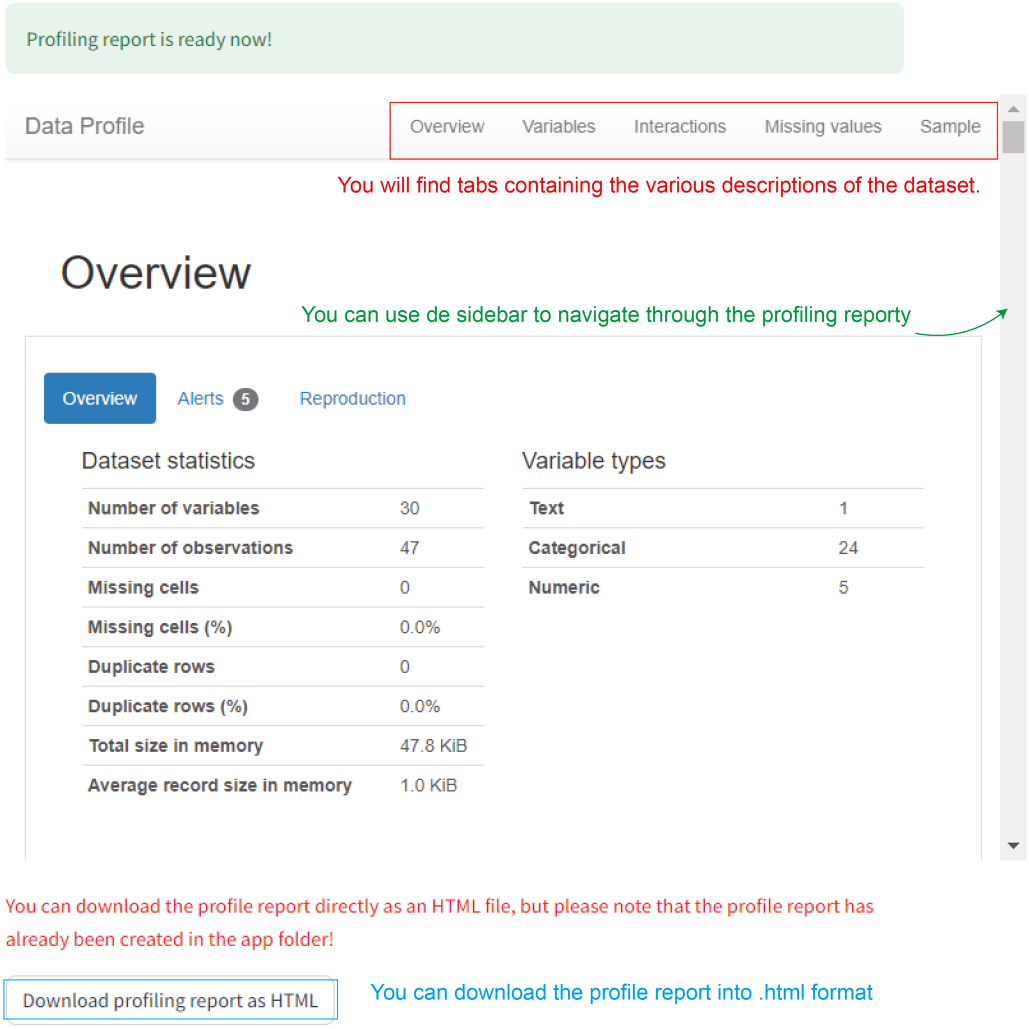


3. On this same page, you can profile your data. Profiling provides you with general information about your data set.
4. Click on Start Profiling to initiate the analysis. Note that the analysis time depends on the size of your data set and your internet connection. This may take several minutes.

You will see this message indicating that the profiling is in progress:



A message will notify you when it is ready:

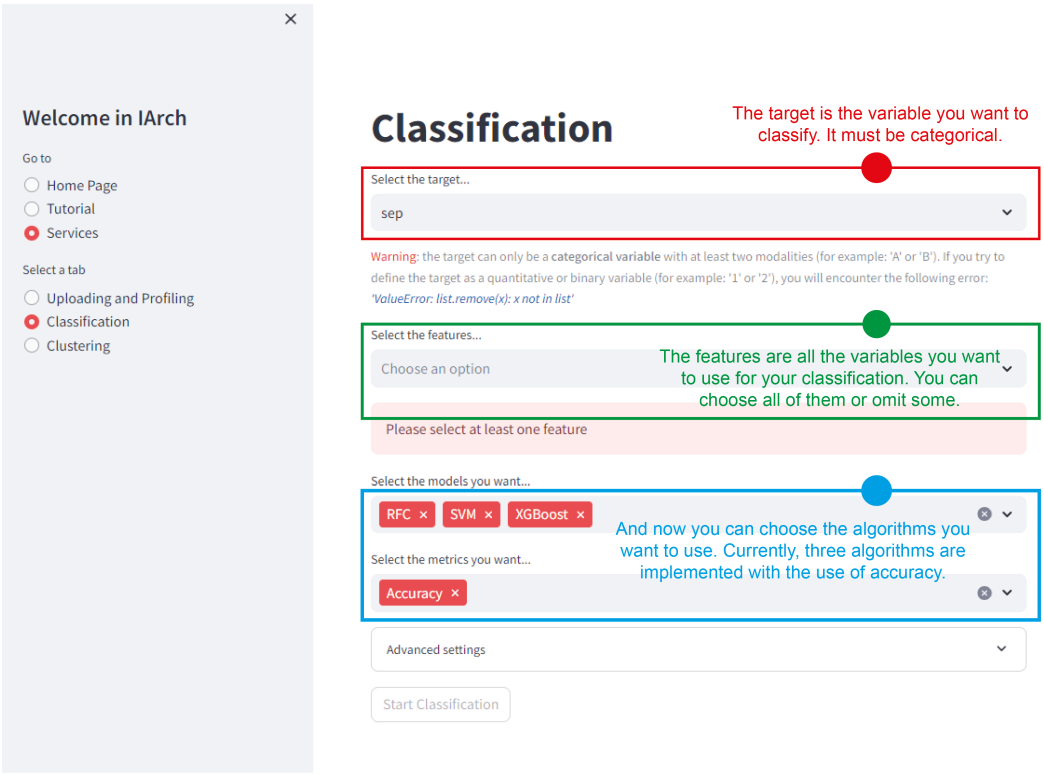


6 Classification Problem

6.1 Functionalities and Services

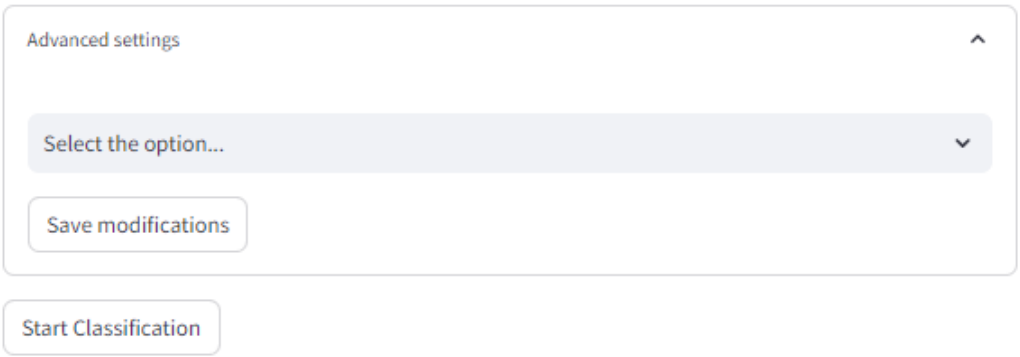
Classification is one of the most common tasks in artificial intelligence (AI) and machine learning. It involves predicting a category or label for new data based on previously labeled examples. This means that the algorithm is trained on a dataset that already contains labels or categories associated with each example (supervised learning). [8, 9, 10]

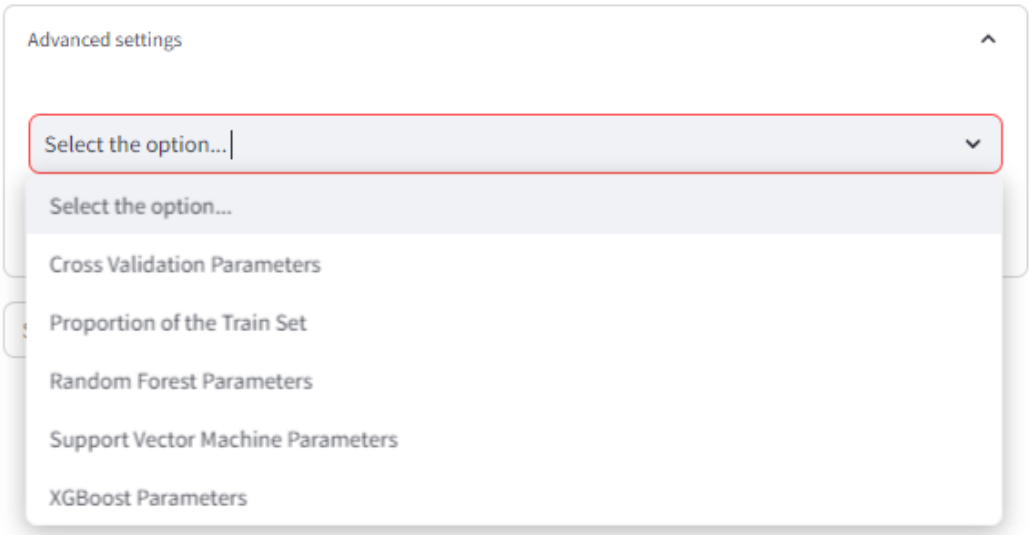
For example, you need to classify the discovered burials based on several criteria. These criteria may include information such as the type of grave, the sex of the individual, the presumed social status, or the historical period to which the burial belongs. The goal is to automatically predict the category of a newly found burial based on the characteristics observed during the excavation. Our application offers you a " *click-based, no code methodology*".



6.2 Advanced settings

You have the option to change the algorithm parameters by clicking on "*Advanced settings*". But be cautious with the choice of parameters.





6.2.1 Cross Validation Parameters

In machine learning, cross-validation is an essential technique to evaluate model performance and select hyper parameters with precision. Cross-validation is used to estimate the performance of a model on unseen data by simulating multiple different splits of the training and validation data. This allows evaluating the ability of the model to generalize to new data. [11]

n_splits

n_splits controls how many times the data is split and used to train and validate the model, thereby influencing the robustness of the model performance estimation. It is typically chosen based on the size of the data set and the desired accuracy of the performance estimate. Common values include 5, 10, or sometimes more, depending on the size of the dataset. Higher n_splits can provide a more reliable estimate of performance, since it uses more data for training and validation. However, this also increases computational cost since the model needs to be trained and evaluated K times. Must be at least 2.

Default = 5

Shuffle

True: The data is randomly shuffled before being divided into training and test sets. Attention, if you change the parameters to "False", you risk a data distribution bias.
Default = True

Cross Validation random state

Random state affects the ordering of the indices, which controls the randomness of each fold. Setting random state to a specific value (such as 0 in this example) ensures the reproducibility of results across executions, as the same random sequence will be used each time. It is not recommended to change this value.
Default = 0

6.2.2 Proprotion of the Train Set

Test size

Test size represents the proportion of your dataset dedicated to test phase. The testing phase then concerns 80 percent of your sample. If you choose a testing proportion greater than 0.3 (equivalent to 70 percent of your dataset in the training phase), you risk poor performance on the training data and insufficient generalization to the test data. In contrast, if you choose a testing proportion less than 0.2 (equivalent to more than 80% of your data set in the training phase), you risk overfitting and a model that does not generalize well.
Default = 0.2
Recommended value = between 0.2 and 0.3

Random state

Setting the "random state" to 0 ensures that the data split will be reproducible every time the code is executed. It is not recommended to change this value.
Default = 0

6.2.3 Random Forest Parameters

Number of estimators

The number of trees in the forest. This parameter specifies how many decision trees the algorithm should create and aggregate to form the final model. For smaller datasets, fewer trees (e.g., 50-200) might be sufficient to achieve good performance. For larger datasets, it is common to use more trees (e.g., 200-500). In cases of very large datasets or when very high accuracy is needed, even more trees (e.g., 500-1000 or more) might be used.
Default = 400

Random Forest Maximum depth

The maximum depth of the tree. It specifies the maximum number of levels that any tree in the forest can grow. A tree grows by splitting nodes based on features of the data, and the maximum depth limits how deep the tree can go. Setting a smaller maximumn depth (3 to 10) results in shallow trees, where each tree makes simpler decisions based on fewer features. Shallow trees tend to be less complex and less prone to overfitting but may underfit the data if set too low. A larger maximum depth (10 to 20 and more) allows the trees to capture more complex relationships in the data, potentially leading to better performance on training data. However, deeper trees can also overfit if the model learns noise in the training data.
For Text recognition = 5-15
For Image recognition = 20 or more

Default = 10

Minimum sample split

It determines the smallest number of samples that a node must have before it can be split into child nodes. If a node has fewer samples than the minimum samples split, it will not be split, and it will become a leaf node. A high value of min_samples_split (20 to 30) implies that each node in the tree requires a substantial number of samples to be eligible for further splitting. High values of min_samples_split can lead to simpler trees with fewer nodes and less detailed decision boundaries. A low value of min_samples_split (2 to 5) allows nodes with fewer samples to be split further, potentially leading to more complex and detailed decision boundaries.

For small to medium dataset = 2 to 20
For Large dataset = 20 to 100
Default = 4

Minimum sample leaf

It specifies the minimum number of samples required to be at a leaf node i.e the endpoints where no further splitting occurs. Each leaf node represents a final decision or prediction. A high value of min_samples_leaf (10 to 20 or more) leads to trees with fewer leaf nodes, potentially simplifying the model by reducing over-fitting and making it less sensitive to noise in the training data. A low value of min_samples_leaf (1 to 5) leads to more complex trees with more leaf nodes. For a complex dataset, you might start with a higher min_samples_leaf value (e.g., 10) to initially simplify the model and prevent over-fitting. For simpler data or when precision is critical, lower min_samples_leaf values (e.g., 1 or 2) might be explored to allow for more detailed models.

Default = 1

Random Forest Random State

Random state affects the ordering of the indices, which controls the randomness of each fold. Setting random state to a specific value (such as 42 here) ensures the reproducibility of results across executions, as the same random sequence will be used each time. The number 42 itself is arbitrary and has no special mathematical significance in this context. It is not recommended to change this value.

Default = 42

6.2.4 Support Vector Machine parameters

C

The parameter C controls the penalty for classification errors in SVM models. It plays a crucial role in determining the margin and robustness of the SVM model. A higher value of implies a higher penalty for classification errors on the training data, which can lead to a narrower margin but better performance on the training data. A lower value of allows for a wider margin, as the model is less sensitive to individual training errors, which can result in better generalization but potentially lower performance on the training data.

Default = 1 000

Kernel

The radial basis function (RBF) kernel measures similarity between two data points in infinite dimensions and then approaches classification by majority vote. It is very flexible and capable of handling complex separations. It is not recommended to modify this parameter.

Default = 'rbf'

Gamma

Gamma is a hyperparameter that defines how much influence a single training example has. The higher the value of gamma, the closer other examples must be to be affected. Low gamma value: This means that the model will have a more generalized decision boundary. Each data point has a large influence, leading to smoother and more generalized decision boundaries. High gamma value: This means that the model will have a more complex decision boundary. Each data point has a small influence, allowing the model to capture more complex patterns but also increasing the risk of overfitting.

Default = 0.01

Probability

The probability option allows for calculating the classification probabilities for each class. This can be useful in cases where you not only need the class prediction but also an estimate of the confidence in that prediction.

Default = True

6.2.5 XGBoost parameters

Number of estimators

This is the number of trees to be created in the XGBoost algorithm. Increasing n_estimators increases the number of trees in the model, which can potentially improve model performance in terms of accuracy. However, too many trees can lead to over-fitting and increase computation time.

Recommended values = 100 - 1000

Default = 200

XGBoost Maximum depth

This is the maximum depth of each decision tree in the algorithm. Deeper trees can capture more complex relationships in the training data, but they can also lead to overfitting if the value is too high.

Recommended values = 3 − 10

Default = 10

Number of columns by tree

This is the fraction of columns to use for constructing each tree. It controls feature subsampling during tree construction. A value less than 1.0 reduces variance and overfitting by introducing more diversity among the trees. If you want to consider all columns/features, choose 1.

Recommended values = 0.3 − 0.8

Default = 1

Learning rate

This is the learning rate that controls the contribution of each tree to correcting the model's prediction errors. It regulates how much each tree adjusts the predictions of the existing model. A lower learning_rate requires more trees to achieve the same performance but can improve model generalization by reducing the risk of overfitting.

Recommended values = 0.01 − 0.3

Default = 0.1

6.3 Example of Results

6.3.1 Data and Options

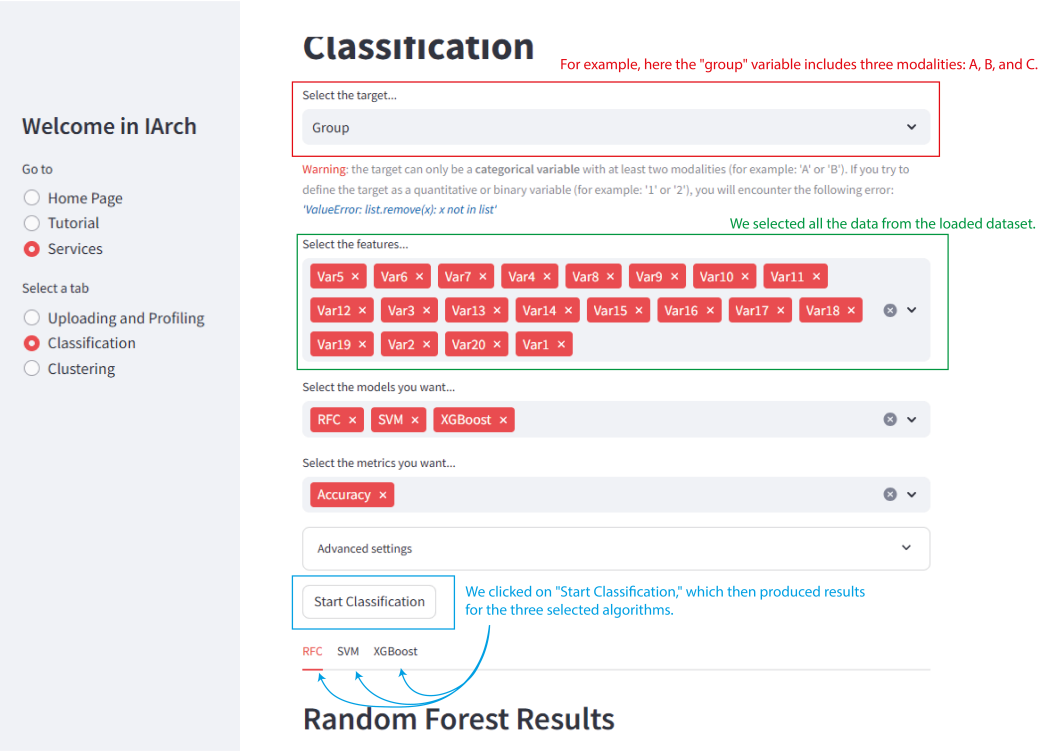
In this example, we created a completely random dataset with 21 variables, including one categorical variable ("group"), which will allow us to solve classification problems.

ID_sep	Group	Var1	Var 2	...	Var20
101	A	0.020877	0.92080	...	0.96951
102	B	0.169090	0.81079	...	0.82046
...	...	...	...	...	...
122	C	0.644562	0.07812	...	0.057936

Table 1: Example of dataset

In the application, we then select the variable group as the "target" variable, and for this example, we include all the variables in the data set while also selecting the three available algorithms.

Once data selection is completed, we can click on "start classification," which will then produce results for each of the algorithms.



6.3.2 Cross-Validation Results

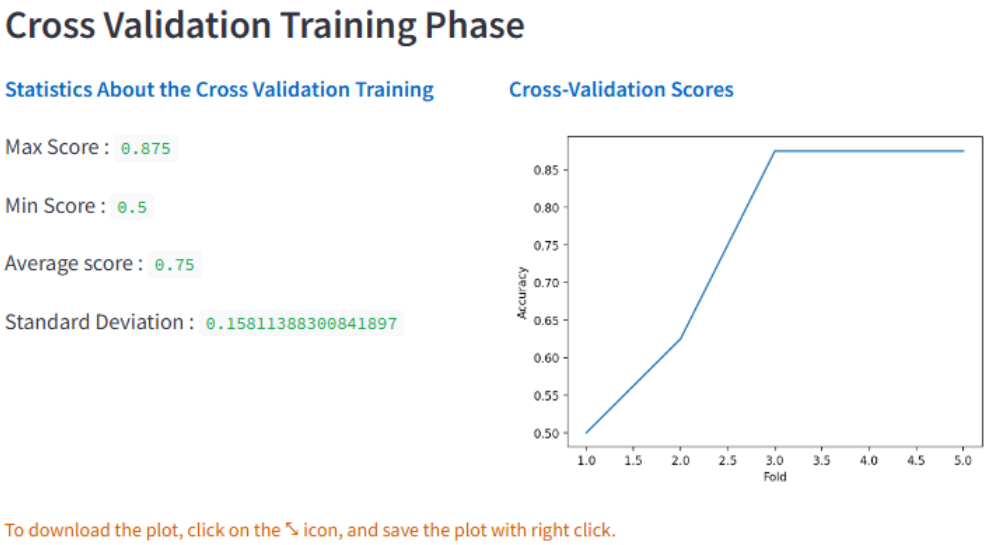
Thus, for each algorithm, we have a summary of a cross-validation for training phase, as numbers and plot. Cross-validation is a technique used to evaluate the performance of a machine learning model by testing it on subsets of data that were not used during training. This provides a more reliable estimate of the model's generalization ability on unseen data.

Max Score (Min score, Average score)

The maximum score indicates the best prediction score achieved, which represents the number of times the algorithm correctly assigned the appropriate group to an individual.

Standard Deviation

Standard deviation is a statistical measure that quantifies the spread or variation of a dataset relative to its mean. Reflects how individual values in a dataset deviate on average from the mean of the dataset. A low standard deviation indicates that the data points are close to the mean, while a high standard deviation indicates greater dispersion among the data points.



6.3.3 Train-Test(80-20) Results

In this section, the results of the training and testing phases are presented:

Training Accuracy

This means that during the training phase, the algorithm was successfully recognized over 75% of the groups of individuals. A relatively high training accuracy indicates that the model has learned well from the training data; however, it is crucial to verify whether it generalizes effectively to new, unseen data.

Test Accuracy

Here, the model correctly classified 80% of the samples in the test set, which is distinct from the training set. This suggests that the model generalizes well and is not merely overfitting to the training data.

Precision

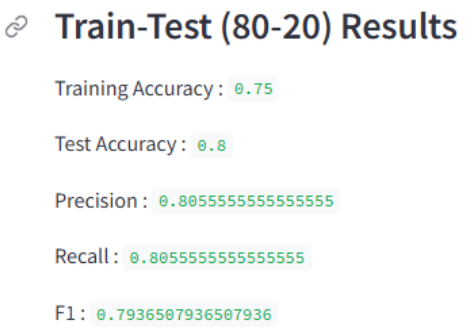
Precision measures the proportion of positive predictions that are actually correct. In other words, out of all the instances that the model predicted as positive, approximately 80.56% were indeed positive. High precision indicates that the model is reliable in classifying positive examples.

Recall

Recall measures the proportion of positive instances that were correctly identified by the model. In other words, out of all the actual positive instances, approximately 80.56% were correctly classified by the model. A high recall is crucial in situations where it is important not to miss positive examples (for instance, in disease detection).

F1

The F1 score is the harmonic mean of precision and recall. It considers both false positives and false negatives, making it particularly useful when there is an imbalance between classes. An F1 score of 79.37% indicates a good balance between precision and recall.



Interpretation of Results:

Overall Performance: The results indicate that the model performs well, with training and testing accuracies of 75% and 80%, respectively. This suggests that the model is capable of correctly generalizing to unseen data.

Balance Between Precision and Recall: The precision and recall values being relatively close indicates that the model does not excessively favor one metric over the other. The F1 score further supports this interpretation by being close to both the precision and recall values.

6.3.4 Explainibility with SHAP

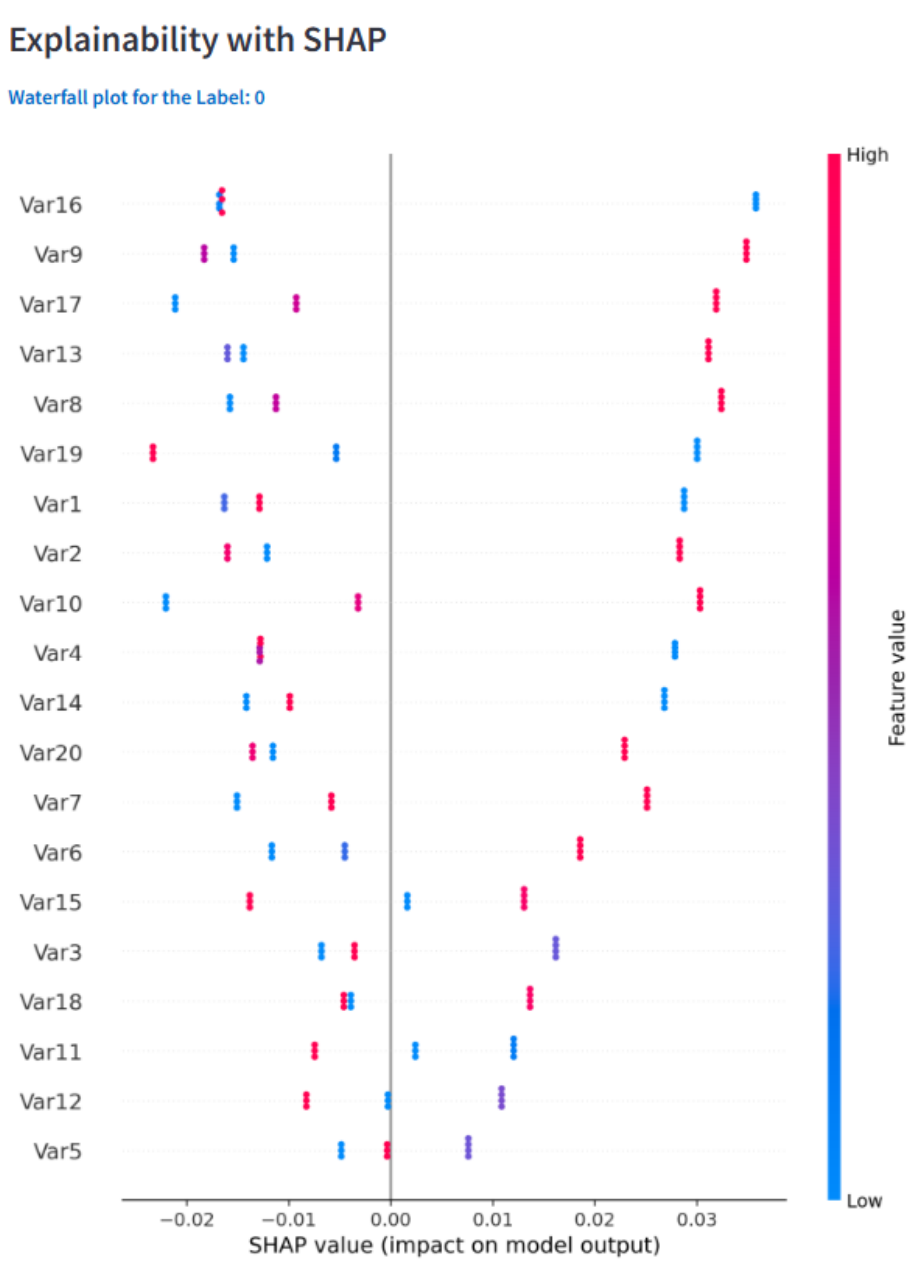
The *Shapley Value library* [5] is generally associated with game theory and machine learning, particularly in the context of model interpretability. It is used to fairly allocate the importance of contributions from various factors or features to the output of a model.

In the field of machine learning, the Shapley value is used to explain model predictions. Complex models, such as random forests or neural networks, can be difficult to interpret. The Shapley value provides a method for understanding the importance of each feature in a specific prediction. For each prediction made by a model, the Shapley value assigns a value to each feature that represents its importance. This allows us to identify which features contributed the most to the model's decision.

The Shapley value ensures that feature contributions are assigned fairly, taking into account interactions between features. It is robust to changes in the data, as it considers all possible permutations.

In summary, the Shapley value is a powerful tool for interpreting machine learning models. It helps make models more transparent by explaining the contribution of each feature to the decision-making process. With libraries like SHAP, it has become accessible to practitioners seeking to better understand and justify their model predictions.

The library creates as many plots as there are classification groups. In our example, there are three groups (A, B, and C), resulting in 3 corresponding plots being generated.



This graph displays the importance of features and their impact on predictions. Each point represents a data instance, with features on the vertical axis and SHAP values on the horizontal axis.

Position: The position of each point along the horizontal axis indicates the effect of that feature on the prediction for that specific instance. Points to the right indicate a positive contribution to the prediction, while points to the left indicate a negative contribution.

Color: The color of the points (a gradient from blue to red) represents the value of the feature for each observation (for example, a red color indicates a high value of the feature). This color coding helps to visualize the relationship between the feature values and their contributions to the predictions, allowing for an easier interpretation of how different values of the feature affect the model's output.

7 Clustering

7.1 Principle, Functionalities and Services

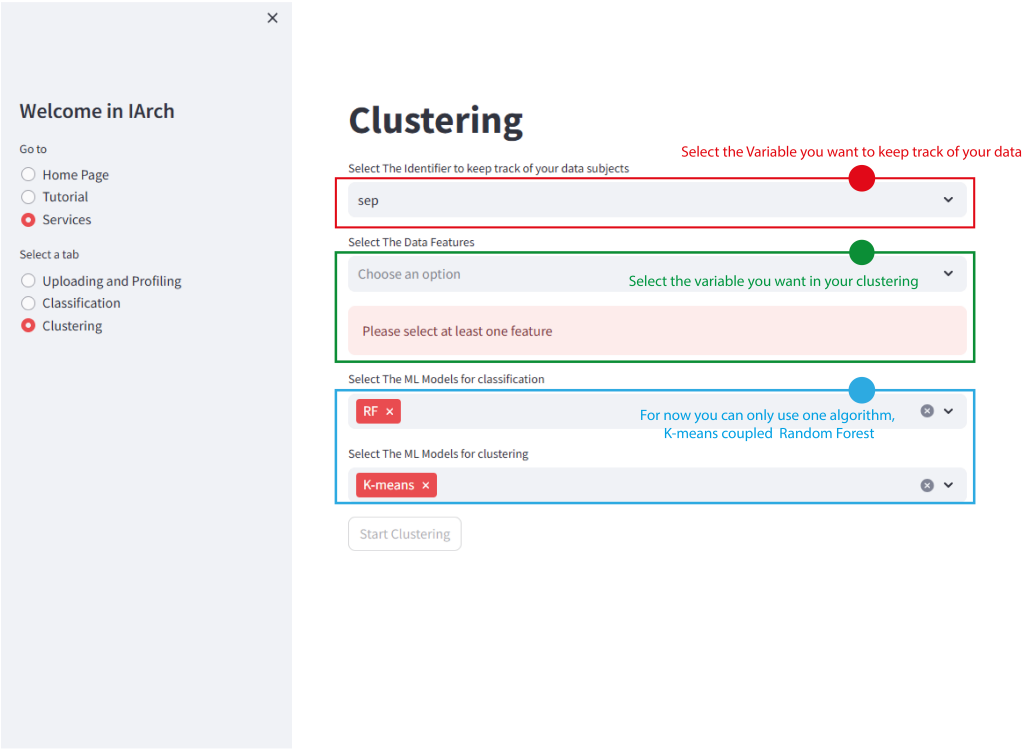
The principle of clustering (or unsupervised classification) involves grouping individuals, graves, or other study elements into several groups called clusters. Unlike supervised learning methods, this approach operates without initial hypotheses: No predefined categories are imposed. Clusters are formed solely on the basis of similarities among the available raw data, without any prior knowledge. [12]

The results can be exported as a .csv spreadsheet with an additional column added to define each cluster. The plots generated using the SHAP library can then be used to characterize the cultural pattern of each cluster.

⚠ ⚠

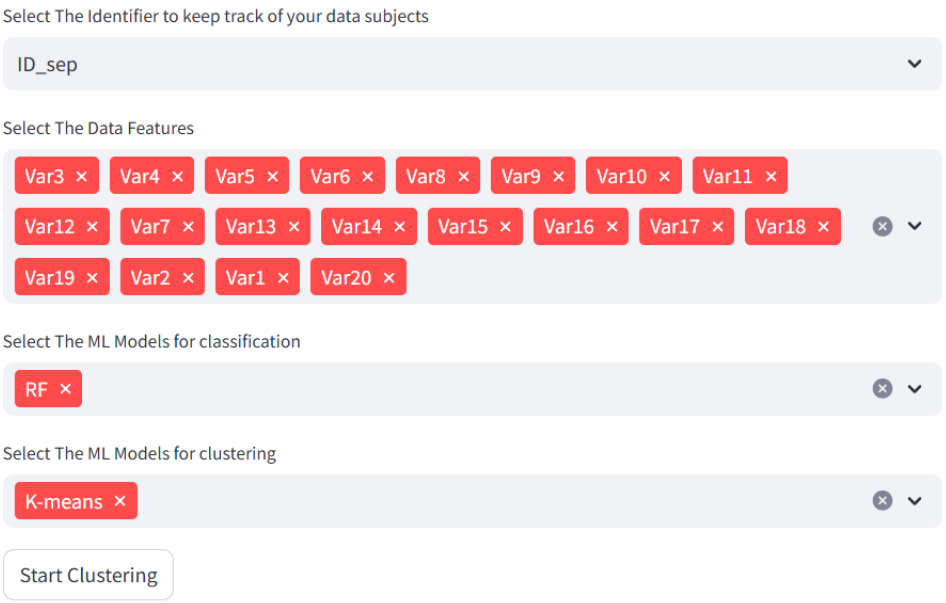
Attention, if you want to perform clustering, please **DO NOT** include the variable of interest in the variable selection. For example, in our case, we do not include the variable 'group' in our selection.

⚠ ⚠



7.2 Example of Results

Using the same dataset, we choose to retain the burial names (e.g., sep102) as a track for result during the export and select all the data from the dataset. Once we have selected all the variables and algorithms (with no other choices available in this version of the application), we can initiate the clustering process.



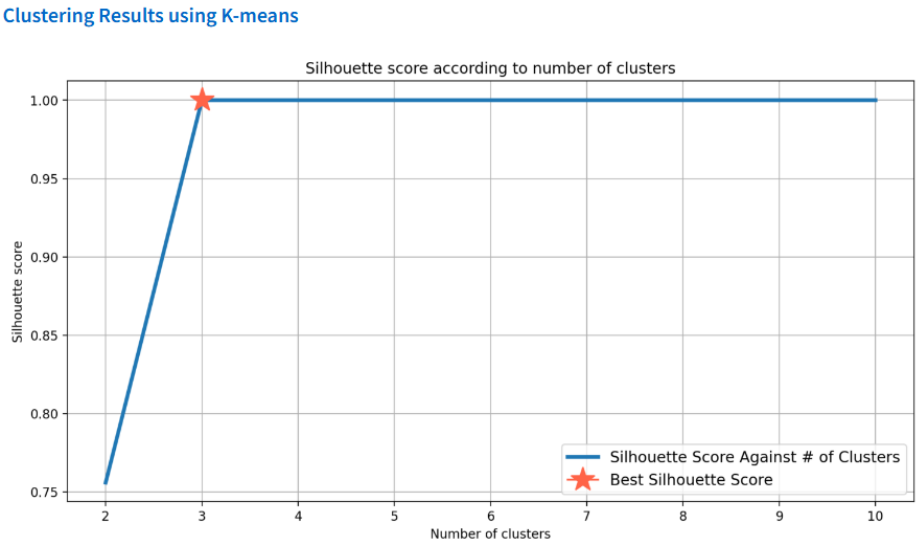
This analysis provides two types of results:

7.2.1 Silhouette Score

The silhouette score is a measure used to evaluate the quality of separation between clusters in a clustering analysis [13]. It provides an indication of how well each data point is clustered with other points in its own cluster compared to other clusters. A score close to 1 indicates that the points are well grouped, while a score close to 0 suggests that the points are on the boundary between two clusters. The silhouette score is particularly useful for determining the optimal number of clusters to use in the K-means algorithm. The number of clusters is chosen to maximize the silhouette score.

However, K-means assumes spherical clusters, which can skew the results if the data does not conform to this shape.

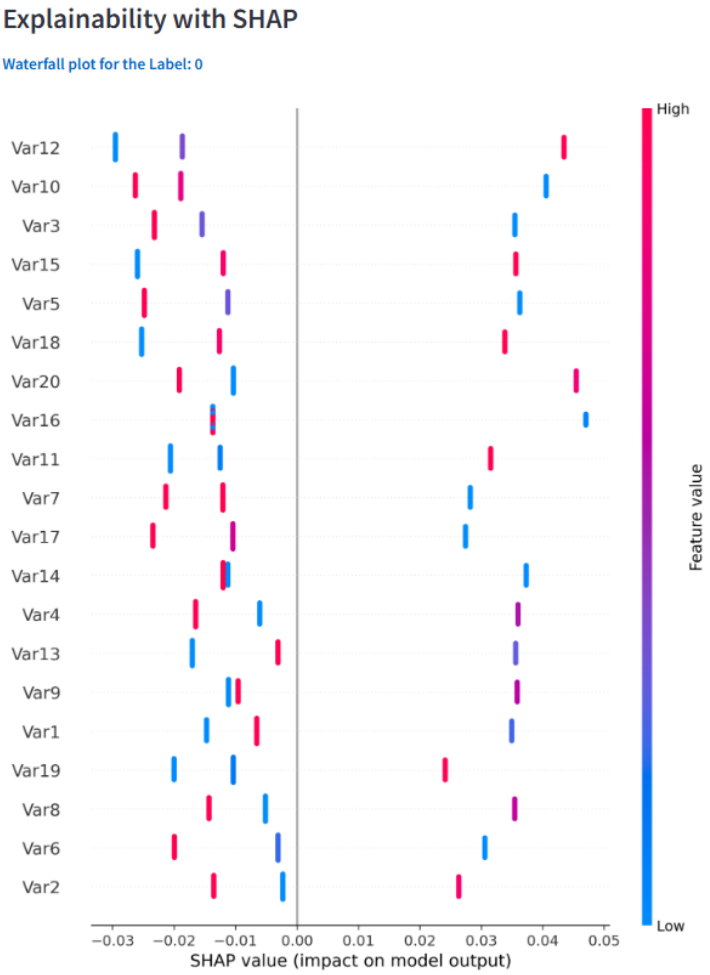
In our example, the clustering produces an optimal number of **3** clusters.



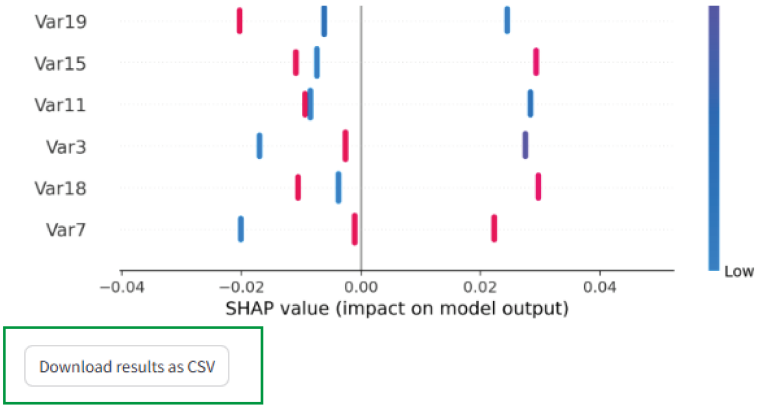
7.2.2 Explainability with SHAP

Explainability with SHAP is exactly the same than for Classification problem. Please refers to 4.3.4 Section (page 16).

The only difference with SHAP in classification is that a graph is produced for each identified cluster (in classification, if you had 3 groups, you would obtain three graphs). If the analysis finds 5 clusters, you will obtain 5 graphs that present the characteristics used in the construction of the clusters.

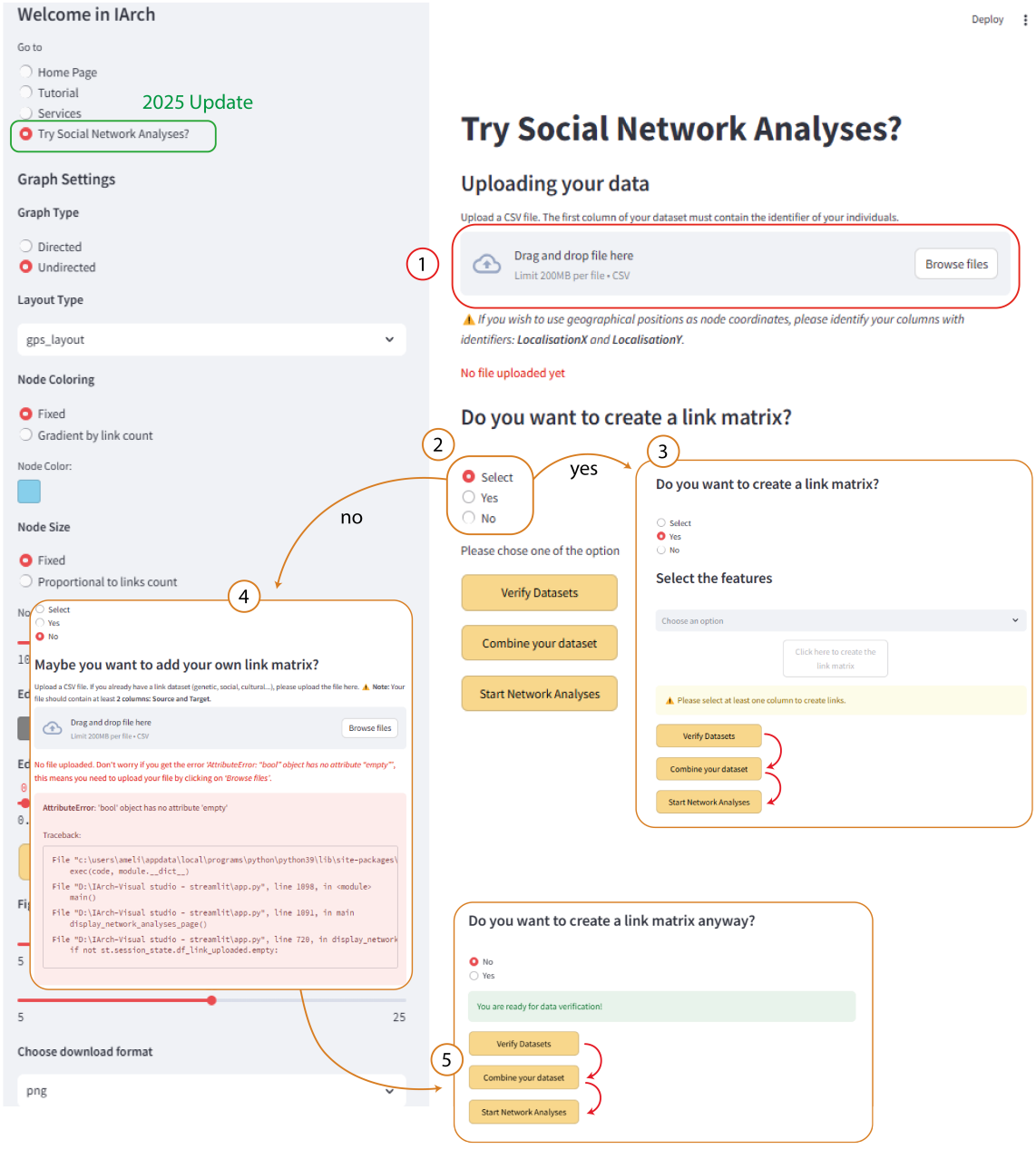


Finally, you have the option to download the dataset in which a column has been added showing the predicted cluster for each individual.



8 2025 UPDATE - Integration of social network analyses (SNA)

8.1 Graph Creation



1. **Upload your dataset**
   Start by uploading your main dataset (e.g., individual information, archaeological features, etc.).

2. **Choose whether to generate a link matrix automatically**
   After uploading the dataset, the app will ask if you want to automatically create a link matrix.

3. **If *yes*: generate the matrix from features**

   - Select the relevant features/columns that will define the links between individuals (e.g., cultural traits, spatial proximity, kinship).

- Click on **"Create the link matrix"**.
- A generated dataset will appear that:
  - Summarizes the number of links created,
  - Shows detailed information for each link.
- Click on **"Combine"** — this step is required to proceed, even if it doesn't change anything here.
- Then click on **"Start Network Analyses"** to launch the SNA.

4. If *no*: you already have a link matrix

- Upload your own link matrix as a `.csv` file.
- The file must include at least two columns: `source` and `target`.
- **Ignore the error message** that appears after uploading — it's expected if the dataset is currently empty.

5. Follow-up confirmation

- After uploading, the app will still ask if you want to generate a link matrix — for example, if you want to add another one (e.g., kinship or cultural links).
- Check the datasets that have been **loaded or created**, even if there's only one.
- Click **"Combine"**, then **"Start Network Analyses"** to run the SNA.

8.2 Graph Visualization

You can configure various options to customize how your social network graph is displayed:

- **Graph type**:
  - **Directed**: links have a direction (e.g., from one individual to another), useful for representing flows or hierarchies such as parent-child relationships.
  - **Undirected**: links are neutral or mutual (e.g., co-occurrence, shared attributes).
- **Graph layout** (node placement):
  - **GPS coordinates**: uses geographical coordinates (requires latitude and longitude columns in your dataset).
  - **Spring layout**: simulates physical repulsion and attraction forces to position nodes in a natural, readable way.
  - **Circular layout**: places all nodes evenly around a circle — useful for small or symbolic networks.
  - **Kamada-Kawai layout**: an algorithm that minimizes energy based on graph-theoretic distances, producing a visually balanced layout.
- **Node color**:
  - **Fixed color**: select any color you like.
  - **Gradient color**: colors nodes according to their number of links (degree).
- **Node size**:
  - **Fixed size**: all nodes have the same size.
  - **Proportional size**: size reflects the number of links for each node.
- **Edge settings**:
  - **Link color**: choose a custom color for the edges.

  - **Link thickness**: set the line width.

- **Exporting the graph**:
  - You can export the graph as an image.
  - Choose the **image size** and the **output format** (e.g., PNG, SVG, PDF).

8.3 Graph Statistics

Once the network is generated, several descriptive statistics are automatically extracted from the graph:

- **Number of nodes**: total number of individuals represented in the network.
- **Number of edges**: total number of connections between individuals.
  Note: If individual A has 4 links with individual B, it is counted as a single edge.
- **Table of the most connected nodes**: displays individuals with the highest number of links (highest degree).
- **Table of the least connected nodes**: shows individuals with the lowest number of links.
- **Table of all nodes and their degree**: a complete list of all individuals along with their number of links.
  This table can be downloaded for further analysis or record-keeping.

9 Conclusion and Perspectives

The IArch © [1] application introduces a powerful tool to streamline and enhance the data analysis process in archaeology and anthropology, bridging a critical gap in post-excavation data interpretation. Through its integration of machine learning and explainable AI techniques, IArch © empowers archaeologists to uncover hidden patterns within datasets, facilitating novel insights that traditional methods might overlook.

The application's robust features offer substantial promise. Its user-oriented interface and thorough tutorial are designed to democratize advanced data analysis, making complex algorithms accessible to professionals without programming expertise. As the development team progresses towards a more refined product, including potential image recognition capabilities and tools for managing missing data, IArch © stands as a foundational step in adapting AI technologies to archaeological research.

The collaborative efforts between the Bird team at LORIA and Toulouse University researchers highlight the dedication to ensuring IArch © meets the specialized demands of archaeological datasets. Future updates and expansions will undoubtedly continue to align the platform with evolving research needs, reinforcing its role as an indispensable resource for archaeologists navigating the challenges of complex data. Through this application, researchers are better equipped to explore, analyze, and interpret their findings with greater depth, precision, and reliability.

10 References

[1] Chahrazed Labba et al. "IArch: An AI Tool for Digging Deeper into Archaeological Data". In: *2023 IEEE 35th International Conference on Tools with Artificial Intelligence (ICTAI)*. Atlanta, United States: IEEE, Nov. 2023, pp. 22–29. DOI: 10.1109/ICTAI59109.2023.00012. URL: https://hal.science/hal-04384735.

[2] I. Cacciari and G. F. Pocobelli. "Machine Learning: A Novel Tool for Archaeology". In: *Handbook of Cultural Heritage Analysis*. Cham: Springer International Publishing, 2022, pp. 961–1002. ISBN: 978-3-030-60016-7. DOI: 10.1007/978-3-030-60016-7_33. URL: https://doi.org/10.1007/978-3-030-60016-7_33.

[3] Simon Bickler. "Machine Learning Arrives in Archaeology". In: *Advances in Archaeological Practice* 9 (May 2021), pp. 186–191. DOI: 10.1017/aap.2021.6.

[4] David Gunning et al. "XAI—Explainable artificial intelligence". In: *Science Robotics* 4.37 (2019), eaay7120. DOI: 10.1126/scirobotics.aay7120. eprint: https://www.science.org/doi/pdf/10.1126/scirobotics.aay7120. URL: https://www.science.org/doi/abs/10.1126/scirobotics.aay7120.

[5] Scott Lundberg and Su-In Lee. *A Unified Approach to Interpreting Model Predictions*. 2017. arXiv: 1705.07874 [cs.AI]. URL: https://arxiv.org/abs/1705.07874.

[6] Microsoft Corporation. *Visual Studio Code, Version 1.91*. https://code.visualstudio.com/. 2023.

[7] Streamlit Inc. *Streamlit, Version 1.31.1*. https://streamlit.io/. 2023.

[8] Michael Berry and Azlinah Mohamed. *Supervised and Unsupervised Learning for Data Science*. Springer Cham, Jan. 2020, p. 191. ISBN: 978-3-030-22474-5. DOI: 10.1007/978-3-030-22475-2.

[9] Sotiris Kotsiantis. "Supervised Machine Learning: A Review of Classification Techniques." In: *Informatica (Slovenia)* 31 (Jan. 2007), pp. 249–268.

[10] Muhammad Iqbal and Zhu Yan. "SUPERVISED MACHINE LEARNING APPROACHES: A SURVEY". In: *International Journal of Soft Computing* 5 (Apr. 2015), pp. 946–952. DOI: 10.21917/ijsc.2015.0133.

[11] Daniel Berrar. “Cross-validation”. In: *Encyclopedia of Bioinformatics and Computational Biology*. Ed. by Shoba Ranganathan et al. Vol. 1. Reference Module in Life Sciences. Elsevier, 2019, pp. 542–545. ISBN: 978-0-12-811432-2. URL: https://oro.open.ac.uk/96776/.

[12] Kristina P. Sinaga and Miin-Shen Yang. “Unsupervised K-Means Clustering Algorithm”. In: *IEEE Access* 8 (2020), pp. 80716–80727. DOI: 10.1109/ACCESS.2020.2988796.

[13] Ketan Rajshekhar Shahapure and Charles Nicholas. “Cluster Quality Analysis Using Silhouette Score”. In: *2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)*. 2020, pp. 747–748. DOI: 10.1109/DSAA49011.2020.00096.